Figure 3: IMDB usecases

fastest open source IMDBs. It is designed to provide high performance on simple SQL queries and DML statements that involve only one table. It supports only limited features, which are used by most real-time applications, like INSERT, UPDATE, DELETE on a single table, and SELECT with local predicates on a single table. It provides multiple interfaces such as JDBC, ODBC and other SQL APIs. CSQL offers atomicity, consistency and isolation. It is typically recommended for use as a cache for existing disk-based commercial databases.

MonetDB: MonetDB is an open source high-performance DBMS developed at the National Research Institute for Mathematics and Computer Science in the Netherlands. It was designed to provide high performance on complex queries against large databases, e.g., combining tables with hundreds of columns and multi-million rows. MonetDB is one of the first database systems to focus its query optimisation effort on exploiting CPU caches. Development of MonetDB started in 1979 and it became an open source project in 2003. MonetDB has been successfully applied in high-performance applications for data mining, OLAP, GIS, XML Query, and text and multimedia retrieval.

How do IMDBs change the rules of the game? While existing applications and schemas can directly benefit from IMDB due to performance improvement, if some things are not carefully handled, the full benefit of IMDB is not achieved. Areas that require significant changes at both the conceptual as well as implementation levels are application design, database schema design and data design (partitioning of data).

Application design

Taking advantage of performance benefits offered by an IMDB means redesigning applications to take advantage of the specific strengths of in-memory technology. The main way to achieve this is to push work that is currently done in the application layer down to the database. This not only allows developers to take advantage of special operations offered by the DBMS, but also reduces the amount of data that must be transferred between application layers. This can lead to substantial performance improvements and can open up new application areas.

Figure 4: IMDB Open Sources

Database schema design

IMDB is beneficial if the data fits into a single database. This requires efforts to conserve space, so more elaborate normalisation procedures are not suitable. Also, using very precise and apt data types enhances storage space. Reduced redundancy, carefully formed columns, precise index creation and efficient data management will help IMDB yield better performance.

Data design

One of the key changes when moving from traditional disk DBs to in-memory DBs is the space available, so data partitioning and storage assumes great significance. IMDB requires as much related data as possible in a single process space. Too much distribution will introduce network I/O in the processing, thereby degrading performance.

Summing up

IMDBs, combined with various current hardware trends, have the ability to change the performance of enterprise and other applications drastically. This, in turn, will result in tremendous value generation to businesses. Such enhanced performance will also foster the evolution of innovative applications and services.

Do send in your feedback and queries, which can be addressed in our forthcoming articles in the IMDB series. END

By: Prasanna Venkatesh
BVP Vamsi

Prasanna has been working on core real-time and HA architectures for over 12 years. He is head of the carrier grade architecture team comprising OS, DB, protocol stacks, DPIBI, Media Cache solutions and security solutions at Huawei. His interests are in distributed databases, telecom protocol stacks, CGL and HA systems. He can be reached at *prasanna.venkatesh@huawei.com*.

Vamsi has been working on in-memory database solutions and fault-tolerance architectures for over 12 years. He has been associated with Huawei for about 11 years and is a lead HA and DB researcher. He can be reached at *vamsi.boyapati@huawei.com*